Middle East Technical University Department of Computer Engineering

CENG 331

Computer Organization

Fall '2025-2026

Performance Lab Homework

Deadline: 8 January 2026, Thursday, 23:59

1 Objectives

This assignment deals with optimizing memory intensive code. Image processing operations offer many examples of functions that can benefit from optimization. In this homework, we will consider two image processing operations that are excellent candidates for cache optimization:

The first operation is bilateral filtering, a non-linear, edge-preserving image filtering technique. Unlike traditional convolution filters that blur edges, bilateral filtering smooths images while preserving sharp edges by considering both spatial proximity and color similarity.

An image is represented as a two-dimensional matrix M, where $M_{i,j}$ denotes the value of the (i, j)th pixel. Pixel values are triples of red, green, and blue (RGB) values. We will only consider square images. Let N denote the number of rows (or columns) of an image. Rows and columns are numbered, in C-style, from 0 to $N - 1$.

The bilateral filter computes a weighted average of neighboring pixels, where the weights depend on both spatial distance and color/intensity difference. For a pixel at position (i, j), the bilateral filter computes:

$$M_{i,j} = \frac{1}{W_{i,j}} \sum_{u=-K}^{K} \sum_{v=-K}^{K} S_{i+u,j+v} \cdot w_s(u, v) \cdot w_r(S_{i,j}, S_{i+u,j+v})$$

where S is the source image, M is the resulting image, K is the filter radius (typically 2 or 3), w_s is the spatial weight function, w_r is the range weight function, and $W_{i,j}$ is the normalization factor. The spatial weight is based on Euclidean distance:

$$w_s(u, v) = \exp\left(-\frac{u^2 + v^2}{2\sigma_s^2}\right)$$

and the range weight is based on color/intensity difference:

$$w_r(S_{i,j}, S_{i+u,j+v}) = \exp\left(-\frac{\|S_{i,j} - S_{i+u,j+v}\|^2}{2\sigma_r^2}\right)$$

where σ_s controls spatial smoothing and σ_r controls edge preservation. For pixels near the image boundaries, we use mirror padding (reflect pixels at boundaries). The bilateral filter is memory-intensive because it requires accessing many neighboring pixels and computing weights for each pixel, making it an ideal candidate for cache optimization techniques such as blocking.

The second operation is box blur, a simple but important image smoothing operation. Box blur computes the average of pixels in a square window around each pixel, creating a smoothing effect. For a pixel at position (i, j), the box blur computes:

$$M_{i,j} = \frac{1}{(2K+1)^2} \sum_{u=-K}^{K} \sum_{v=-K}^{K} S_{i+u,j+v}$$

where S is the source image, M is the resulting image, and K is the filter radius (typically 2, giving a 5x5 window). For pixels near the image boundaries, we use mirror padding (reflect pixels at boundaries). Box blur is memory-intensive because it requires accessing many neighboring pixels for each output pixel, making it a good candidate for cache optimization techniques such as blocking.

2 Specifications

Start by copying `perflab-handout.tar` to a protected directory in which you plan to do your work. Then give the command: `tar xvf perflab-handout.tar`. This will cause a number of files to be unpacked into the directory. The only file you will be modifying and handing in is `kernels.c`. The `driver.c` program is a driver program that allows you to evaluate the performance of your solutions. Use the command `make driver` to generate the driver code and run it with the command `./driver`.

Looking at the file `kernels.c` you'll notice a C structure `team` into which you should insert the requested identifying information about you. **Do this right away so you don't forget.**

3 Implementation Overview

Data Structures

The image is represented as a 2D array of pixels where a `pixel` is a struct defined as:

```
typedef struct {
   unsigned short red;   /* R value */
   unsigned short green; /* G value */
   unsigned short blue;  /* B value */
} pixel;
```

An image `I` is represented as a one-dimensional array of `pixel`s, where the (i, j)th pixel is `I[RIDX(i,j,n)]`. Here `n` is the dimension of the image matrix, and `RIDX` is a macro defined as follows:

```
#define RIDX(i,j,n) ((i)*(n)+(j))
```

See the file `defs.h` for this code.

Bilateral Filter

The bilateral filter function takes as input a source image `src`, and returns the filtered result in the destination image `dst`. The filter radius `radius`, spatial parameter `sigma_s`, and range parameter `sigma_r` it needs are globally defined. Here is a naive implementation:

```
static int radius = 2;  /* Filter radius (window size is 2*radius+1) */
static double sigma_s = 2.0;  /* Spatial parameter */
static double sigma_r = 50000.0;  /* Range parameter */

char naive_bilateral_descr[] = "naive_bilateral: Naive baseline implementation";
void naive_bilateral(int dim, pixel *src, pixel *dst)
{
    int i, j, u, v;
    int src_i, src_j;
    double sum_r, sum_g, sum_b;
    double weight_sum;
    double spatial_weight, range_weight, weight;
    double dist_sq, color_diff_sq;
    double two_sigma_s_sq = 2.0 * sigma_s * sigma_s;
    double two_sigma_r_sq = 2.0 * sigma_r * sigma_r;

    for(i = 0; i < dim; i++) {
        for(j = 0; j < dim; j++) {
            sum_r = sum_g = sum_b = 0.0;
            weight_sum = 0.0;

            for(u = -radius; u <= radius; u++) {
                for(v = -radius; v <= radius; v++) {
                    src_i = i + u;
                    src_j = j + v;

                    /* Mirror padding at boundaries */
                    if(src_i < 0) src_i = -src_i;
                    if(src_i >= dim) src_i = 2*dim - 1 - src_i;
                    if(src_j < 0) src_j = -src_j;
                    if(src_j >= dim) src_j = 2*dim - 1 - src_j;

                    /* Spatial weight */
                    dist_sq = u*u + v*v;
                    spatial_weight = exp(-dist_sq / two_sigma_s_sq);

                    /* Range weight (using L2 norm of RGB difference) */
                    color_diff_sq = ((double)src[RIDX(i, j, dim)].red -
                                     (double)src[RIDX(src_i, src_j, dim)].red) *
                                    ((double)src[RIDX(i, j, dim)].red -
                                     (double)src[RIDX(src_i, src_j, dim)].red) +
                                    ((double)src[RIDX(i, j, dim)].green -
                                     (double)src[RIDX(src_i, src_j, dim)].green) *
                                    ((double)src[RIDX(i, j, dim)].green -
                                     (double)src[RIDX(src_i, src_j, dim)].green) +
                                    ((double)src[RIDX(i, j, dim)].blue -
                                     (double)src[RIDX(src_i, src_j, dim)].blue) *
                                    ((double)src[RIDX(i, j, dim)].blue -
                                     (double)src[RIDX(src_i, src_j, dim)].blue);
```

```
                    range_weight = exp(-color_diff_sq / two_sigma_r_sq);
                    weight = spatial_weight * range_weight;
                    weight_sum += weight;
                    sum_r += src[RIDX(src_i, src_j, dim)].red * weight;
                    sum_g += src[RIDX(src_i, src_j, dim)].green * weight;
                    sum_b += src[RIDX(src_i, src_j, dim)].blue * weight;
                }
            }
            dst[RIDX(i, j, dim)].red = (unsigned short)(sum_r / weight_sum);
            dst[RIDX(i, j, dim)].green = (unsigned short)(sum_g / weight_sum);
            dst[RIDX(i, j, dim)].blue = (unsigned short)(sum_b / weight_sum);
        }
    }
}
```

The naive implementation has poor cache locality because it accesses many different regions of the source image for each output pixel and performs expensive weight calculations. Your task is to optimize `bilateral` to run as fast as possible using cache blocking techniques and efficient weight computation. This code is in the file `kernels.c`.

Box Blur

The box blur function takes a source image `src` and returns the blurred result in the destination image `dst`. Here is a naive implementation:

```
void naive_blur(int dim, pixel *src, pixel *dst) {
    int i, j, u, v;
    int src_i, src_j;
    int sum_r, sum_g, sum_b;
    int radius = 2;
    int window_size = 2 * radius + 1;
    int window_area = window_size * window_size;

    for(i = 0; i < dim; i++) {
        for(j = 0; j < dim; j++) {
            sum_r = sum_g = sum_b = 0;

            for(u = -radius; u <= radius; u++) {
                for(v = -radius; v <= radius; v++) {
                    src_i = i + u;
                    src_j = j + v;

                    /* Mirror padding at boundaries */
                    if(src_i < 0) src_i = -src_i;
                    if(src_i >= dim) src_i = 2*dim - 1 - src_i;
                    if(src_j < 0) src_j = -src_j;
                    if(src_j >= dim) src_j = 2*dim - 1 - src_j;

                    sum_r += src[RIDX(src_i, src_j, dim)].red;
                    sum_g += src[RIDX(src_i, src_j, dim)].green;
                    sum_b += src[RIDX(src_i, src_j, dim)].blue;
                }
            }

            dst[RIDX(i, j, dim)].red = (unsigned short)(sum_r / window_area);
            dst[RIDX(i, j, dim)].green = (unsigned short)(sum_g / window_area);
            dst[RIDX(i, j, dim)].blue = (unsigned short)(sum_b / window_area);
        }
    }
}
```

The naive implementation has poor cache locality because it accesses many different regions of the source image for each output pixel. Your task is to optimize `blur` to run as fast as possible using cache blocking techniques. This code is in the file `kernels.c`.

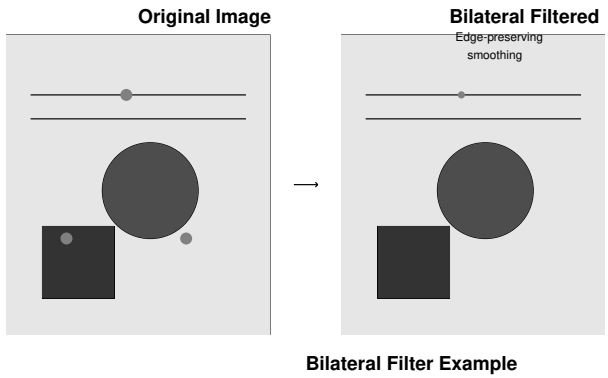

Figure 1: Example of bilateral filtering: (left) original image, (right) filtered image showing edge-preserving smoothing.

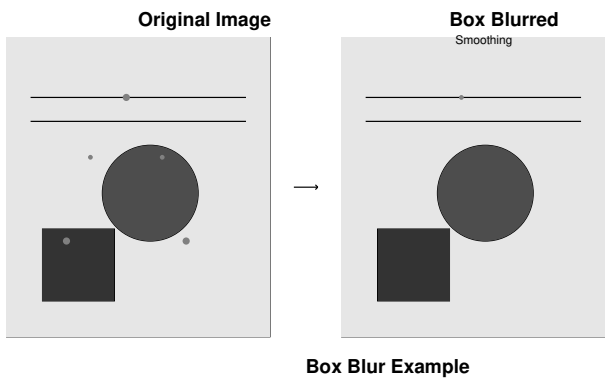

Figure 2: Example of box blur: (left) original image, (right) blurred image showing smoothing effect.

Performance measures

Our main performance measure is *CPE* or *Cycles per Element*. If a function takes C cycles to run for an image of size $N \times N$, the CPE value is C/N^2. Table 1 summarizes the performance of the naive implementations shown above and compares it against an optimized implementation. Performance is shown for for 5 different values of N. All measurements were made on the the department computers (ineks).

The ratios (speedups) of the optimized implementation over the naive one will constitute a *score* of your implementation. To summarize the overall effect over different values of N, we will compute the *geometric mean* of the results for these 5 values. That is, if the measured speedups for $N = \{64, 128, 256, 512, 1024\}$ are R_{64}, R_{128}, R_{256}, R_{512}, and R_{1024} then we compute the overall performance as

$$R = \sqrt[5]{R_{64} \times R_{128} \times R_{256} \times R_{512} \times R_{1024}}$$

Test case	1	2	3	4	5	
Method N	64	128	256	512	1024	Geom. Mean
Naive `bilateral` (CPE)	1442.9	1440.9	1447.5	1448.6	1451.9	
Optimized `bilateral` (CPE)	907.3	906.9	910.1	911.1	913.6	
Speedup (naive/opt)	1.6	1.6	1.6	1.6	1.6	1.6
Method N	64	128	256	512	1024	Geom. Mean
Naive `blur` (CPE)	103.6	103.5	103.5	103.8	104.5	
Optimized `blur` (CPE)	71.9	72.0	72.4	72.2	72.9	
Speedup (naive/opt)	1.4	1.4	1.4	1.4	1.4	1.4

Table 1: CPEs and Ratios for Optimized vs. Naive Implementations

Assumptions

To make life easier, you can assume that N is a multiple of 32. Your code must run correctly for all such values of N, but we will measure its performance only for the 5 values shown in Table 1.

4 Infrastructure

We have provided support code to help you test the correctness of your implementations and measure their performance. This section describes how to use this infrastructure. The exact details of each part of the assignment is described in the following section.

Note: The only source file you will be modifying is `kernels.c`.

Versioning

You will be writing many versions of the `bilateral` and `blur` routines. To help you compare the performance of all the different versions you've written, we provide a way of "registering" functions.

For example, the file `kernels.c` that we have provided you contains the following functions:

```
void register_bilateral_functions() {
   add_bilateral_function(&bilateral, bilateral_descr);
}

void register_blur_functions() {
   add_blur_function(&blur, blur_descr);
}
```

This function contains one or more calls to `add_bilateral_function` and `add_blur_function`. In one of the examples above,
`add_bilateral_function` registers the function `bilateral` along with a string `bilateral_descr` which is an ASCII description of what the function does. See the file `kernels.c` to see how to create the string descriptions. This string can be at most 256 characters long. Blur works the same way.

Driver

The source code you will write will be linked with object code that we supply into a `driver` binary. To create this binary, you will need to execute the command

```
unix> make driver
```

You will need to re-make driver each time you change the code in `kernels.c`. To test your implementations, you can then run the command:

```
unix> ./driver
```

The `driver` can be run in four different modes:

- *Default mode*, in which all versions of your implementation are run.
- *Autograder mode*, in which only the `bilateral()` and `blur()` functions are run. This is the mode we will use when grading your solution.
- *File mode*, in which only versions that are mentioned in an input file are run.
- *Dump mode*, in which a one-line description of each version is dumped to a text file. You can then edit this text file to keep only those versions that you'd like to test using the *file mode*. You can specify whether to quit after dumping the file or if your implementations are to be run.

If run without any arguments, `driver` will run all of your versions (*default mode*). Other modes and options can be specified by command-line arguments to `driver`, as listed below:

- `-g` : Run only `bilateral()` and `blur()` functions (*autograder mode*).
- `-f <funcfile>` : Execute only those versions specified in `<funcfile>` (*file mode*).
- `-d <dumpfile>` : Dump the names of all versions to a dump file called `<dumpfile>`, *one line* to a version (*dump mode*).
- `-q` : Quit after dumping version names to a dump file. To be used in tandem with `-d`. For example, to quit immediately after printing the dump file, type `./driver -qd dumpfile`.
- `-h` : Print the command line usage.

Team Information

Important: Before you start, you should fill in the struct in `kernels.c` with information about your team (team name, student names, student ids).

5 Assignment Details

Optimizing Bilateral Filter (70 points)

In this part, you will optimize `bilateral` to achieve as low a CPE as possible. The key challenge is improving cache locality by using blocking techniques to keep frequently accessed image regions in cache, while efficiently computing spatial and range weights.

For example, running driver with the supplied naive version (for `bilateral`) generates the output shown below:

```
unix> ./driver
Teamname: Team
Member 1: Student Name
ID 1: eXXXXXXX

Bilateral: Version = naive_bilateral: Naive baseline implementation:
Dim             64      128     256     512     1024    Mean
Your CPEs       1443.1  1443.6  1449.8  1449.6  1452.6
Baseline CPEs   1442.9  1440.9  1447.5  1448.6  1451.9
Speedup         1.0     1.0     1.0     1.0     1.0     1.0
```

Optimizing Box Blur (30 points)

In this part, you will optimize `blur` to achieve as low a CPE as possible. You should compile `driver` and then run it with the appropriate arguments to test your implementations. While simpler than bilateral filtering, box blur still benefits from cache blocking techniques to improve spatial locality when accessing neighboring pixels.

For example, running driver with the supplied naive version (for `blur`) generates the output shown below:

```
unix> ./driver
Teamname: Team
Member 1: Student Name
ID 1: eXXXXXXX

...

...


Blur: Version = naive_blur: Naive baseline implementation:
Dim             64      128     256     512     1024    Mean
Your CPEs       103.6   103.5   102.8   103.4   104.4
Baseline CPEs   103.6   103.5   103.5   103.8   104.5
Speedup         1.0     1.0     1.0     1.0     1.0     1.0
```

Some advice. Look at the assembly code generated for the `bilateral` and `blur`. For bilateral filtering, focus on cache blocking techniques to improve spatial locality when accessing neighboring pixels, and consider precomputing spatial weights or using lookup tables for exponential functions. For box blur, focus on cache blocking techniques to improve spatial locality when accessing neighboring pixels in the 5x5 window. These optimization tricks are covered in class and in the textbook.

Coding Rules

You may write any code you want, as long as it satisfies the following:

- It must be in ANSI C. You may not use any embedded assembly language statements.
- It must not interfere with the time measurement mechanism. You will also be penalized if your code prints any extraneous information.

You can only modify code in `kernels.c`. You are allowed to define macros, additional global variables, and other procedures in this file.

Evaluation

- Correctness: You will get NO CREDIT for buggy code that causes the driver to complain! This includes code that correctly operates on the test sizes, but incorrectly on image matrices of other sizes. As mentioned earlier, you may assume that the image dimension is a multiple of 32.
- Note that you should only modify `dst` pointer. If you modify input pointers or exceed their limits, you will not get any credit.
- Bilateral Filter: You will get 40 points for implementation of `bilateral` if it is correct and achieve mean speed-up threshold of 1.3. The team that achieves the biggest speed-up will get 70 points. Other grades will be scaled between 40 and 70 according to your speed-up. You will not get any partial credit for a correct implementation that does below the threshold.
- Box Blur: You will get 30 points if your implementation is correct and achieve a mean speed-up of 1.3. There is no scaling in this function.
- Since there might be changes of performance regarding to CPU status, test the same code many times and take only the best into consideration. When your codes are evaluated, your codes will be tested in a closed environment many times and only your best speed-up of functions will be taken into account.
- Speedups will be measured on department computers(inek).
- Optional: In this assignment, you have an option to create a team up to three people. However, you can also do it alone.

Submission

Submissions are done via ODTUClass. You can only submit `kernels.c` function. Therefore make sure all of your changes are only on this file. Any member of the team can submit the file. Do **NOT** submit the same file by different members of the team. One submission is enough.